

LABORATORY RECORD

PYTHON AUTOMATION

Robot Framework — Keyword-Driven & Data-Driven Test Automation

Submitted by

Pratyay Chowdhury

Enrollment No.: 12023052020034

B.Tech, Computer Science & Engineering (AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT

Department of Computer Science & Engineering (AI)

Kolkata, West Bengal

Index

Sl. No.	Experiment Title	Page
13	Basic Syntax and Keywords	3
14	Variables and Data-Driven Automation	5
15	Custom Keywords and Libraries	7
16	Assertions and Verification	9
17	Test Setup and Teardown	11
18	Tags and Test Execution	13
19	Reports and Logs	15

Introduction to Robot Framework Automation

Robot Framework is a generic, keyword-driven test automation framework where test cases are written as readable keywords instead of raw programming statements. A test case such as "Input Text id:user-name standard_user" reads almost like an instruction to a human tester, while under the hood each keyword is backed by a library — SeleniumLibrary for browser actions, RequestsLibrary for API calls, or a custom Python library for project-specific logic. This makes Robot Framework popular for teams that want automation scripts reviewable by non-programmers without sacrificing full programmatic power underneath.

This record covers Milestone 4 of the syllabus — Robot Framework — through seven experiments spanning the full breadth of the module's lab assignment: basic syntax and keywords, variables and data-driven automation, custom keywords and libraries, assertions and verification, test setup/teardown, tags and selective execution, and HTML reports and logs. Each experiment follows the structure prescribed in the syllabus — problem statement, objective, tools used, implementation, output and conclusion.

Common Environment Setup

The following setup applies to all experiments in this record unless stated otherwise:

- Python 3.10+ with pip
- robotframework, robotframework-selenium library, robotframework-requests, robotframework-data driver, robotframework-pabot
- IDE: RIDE (Robot IDE) or PyCharm with the Robot Framework plugin
- Google Chrome browser installed
- Target web UI: <https://www.saucedemo.com>
- Target REST API: <https://reqres.in>

Experiment 13: Basic Syntax and Keywords

Module	Robot Framework — Introduction (3.1)
Experiment Title	Basic Syntax and Keywords

1. Problem Statement

Teams new to Robot Framework need to see its keyword-driven syntax in action before building anything larger: opening a browser, filling in a form field, and checking that an element exists on the page, all expressed as plain keywords rather than programming statements.

2. Objective

- Write a Robot Framework test case that opens a browser and navigates to a specific URL.
- Use the Input Text keyword to fill in a form field on the page.
- Use the Page Should Contain Element keyword to verify a specific element is present.

3. Tools, Software and Concepts Used

- Robot Framework, Selenium Library
- Keywords: Open Browser, Input Text, Click Button, Page Should Contain Element

- Target site: <https://www.saucedemo.com>

4. Implementation Steps

- Install Robot Framework and Selenium Library via pip.
- Create login_basic.robot with a ***** Settings ***** section importing Selenium Library.
- Declare the target URL and browser name as suite variables.
- In the ***** Test Cases ***** section, open the browser, fill the username and password fields with Input Text, click the login button, and assert the inventory container element is present.
- Run the suite from the terminal with the robot command.

5. Source Code

```
login_basic.robot

1  *** Settings ***
2  Library      SeleniumLibrary
3
4  *** Variables ***
5  ${URL}       https://www.saucedemo.com
6  ${BROWSER}   Chrome
7
8  *** Test Cases ***
9  Open Browser And Fill Login Form
10     Open Browser    ${URL}    ${BROWSER}
11     Title Should Be    Swag Labs
12     Input Text      id:user-name    standard_user
13     Input Text      name:password    secret_sauce
14     Click Button     id:login-button
15     Page Should Contain Element    id:inventory_container
16     [Teardown]      Close Browser
```

6. Output

```
Terminal

$ robot login_basic.robot

=====
Login_Basic
=====
Open Browser And Fill Login Form                                | PASS |
-----
Login_Basic                                                    | PASS |
1 test, 1 passed, 0 failed
=====
Output:  output.xml
Log:     log.html
Report:  report.html
Process finished with exit code 0
```

Fig 13.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Open Browser, Input Text and Page Should Contain Element read almost like plain instructions, yet each one executes a real Selenium action underneath. This confirmed a beginner-level Robot Framework suite can automate a full navigate-fill-verify flow without writing a single line of raw Python, which is the core value proposition of a keyword-driven framework.

Experiment 14: Variables and Data-Driven Automation

Module	Robot Framework — Introduction (3.1)
Experiment Title	Variables and Data-Driven Automation

1. Problem Statement

Hardcoding a username and password directly inside a test case ties the credentials to that one test and makes them awkward to reuse or change. Beyond that, the login flow itself should be verifiable against several different user accounts without duplicating the test case for each one.

2. Objective

- Implement a test case that uses Robot Framework variables to store values like username and password.
- Create a data-driven test that reads test data from an external file and performs the same steps for every row.

3. Tools, Software and Concepts Used

- Robot Framework, SeleniumLibrary
- DataDriver library (robotframework-datadrivers) for external CSV data
- Test Template keyword
- Target site: <https://www.saucedemo.com>

4. Implementation Steps

- Move the username and password into the `*** Variables ***` section (or an external variable file) instead of inline strings.
- Create `login_data.csv` with columns `UserName`, `Password` and `Expected`.
- Import the DataDriver library, pointing it at `login_data.csv`, and set Test Template to the keyword that performs the login attempt.
- Write a single templated test case row so DataDriver expands it into one run per CSV row.
- Define the Attempt Login keyword that receives the row's arguments and checks the expected outcome text on the page.

5. Source Code

```
login_ddt.robot + login_data.csv

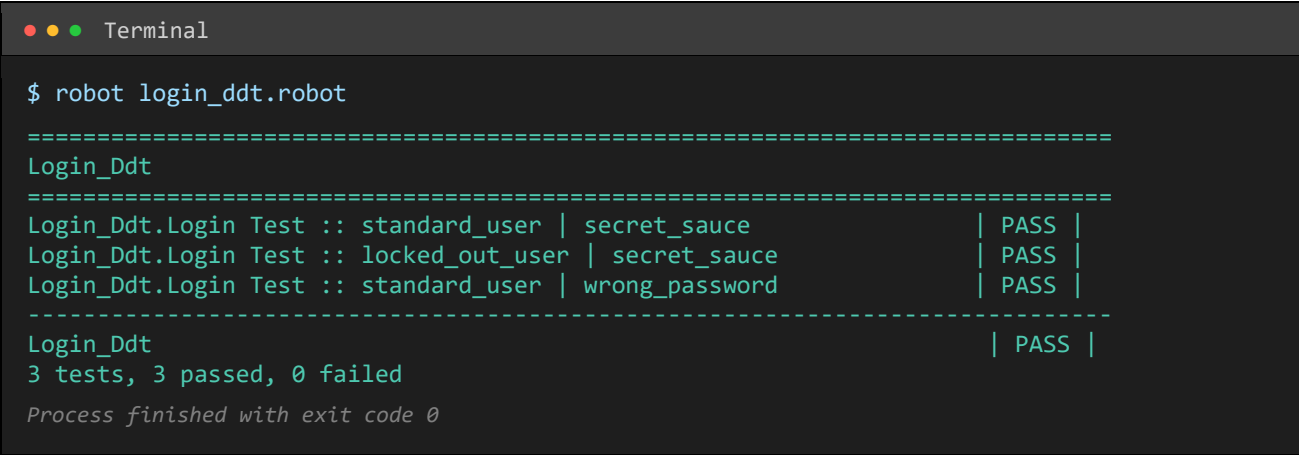
1  *** Settings ***
2  Library      SeleniumLibrary
3  Library      DataDriver      file=login_data.csv
4  Test Template  Attempt Login
5
6  *** Variables ***
7  ${URL}        https://www.saucedemo.com
8  ${BROWSER}    Chrome
9
10 *** Test Cases ***
11 Login Test    Default    UserName    Password    Expected
12
13 *** Keywords ***
14 Attempt Login
15     [Arguments]    ${UserName}    ${Password}    ${Expected}
```

```

16      Open Browser      ${URL}      ${BROWSER}
17      Input Text        id:user-name  ${UserName}
18      Input Text        name:password  ${Password}
19      Click Button      id:login-button
20      Page Should Contain  ${Expected}
21      [Teardown]        Close Browser
22
23      # login_data.csv
24      # UserName,Password,Expected
25      # standard_user,secret_sauce,Products
26      # locked_out_user,secret_sauce,locked out
27      # standard_user,wrong_password,do not match

```

6. Output



```

Terminal
$ robot login_ddt.robot

=====
Login_Ddt
=====
Login_Ddt.Login Test :: standard_user | secret_sauce           | PASS |
Login_Ddt.Login Test :: locked_out_user | secret_sauce         | PASS |
Login_Ddt.Login Test :: standard_user | wrong_password        | PASS |
-----
Login_Ddt                                                    | PASS |
3 tests, 3 passed, 0 failed
Process finished with exit code 0

```

Fig 14.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Pulling the credentials into variables made the login keyword reusable across scenarios, and the DataDriver library expanded a single templated test case into three independent runs straight from login_data.csv. Every combination showed up as its own named result in the report, which gives the same visibility as writing three separate test cases while keeping the actual steps defined only once.

Experiment 15: Custom Keywords and Libraries

Module	Robot Framework — Introduction (3.1)
Experiment Title	Custom Keywords and Libraries

1. Problem Statement

Some actions — like a project-specific calculation — have no ready-made Robot Framework keyword. The framework needs to be extended with a custom Python library, and existing built-in capabilities (like BuiltIn's string and math helpers) should be used before writing new code.

2. Objective

- Build a custom Robot Framework keyword in Python that calculates the sum of two numbers.
- Use the BuiltIn library to manipulate strings or perform mathematical operations directly inside a test case.

3. Tools, Software and Concepts Used

- Robot Framework, Python 3
- A custom Python library file exposing keywords automatically
- BuiltIn library (Evaluate, Convert To Integer, Should Be Equal As Integers)

4. Implementation Steps

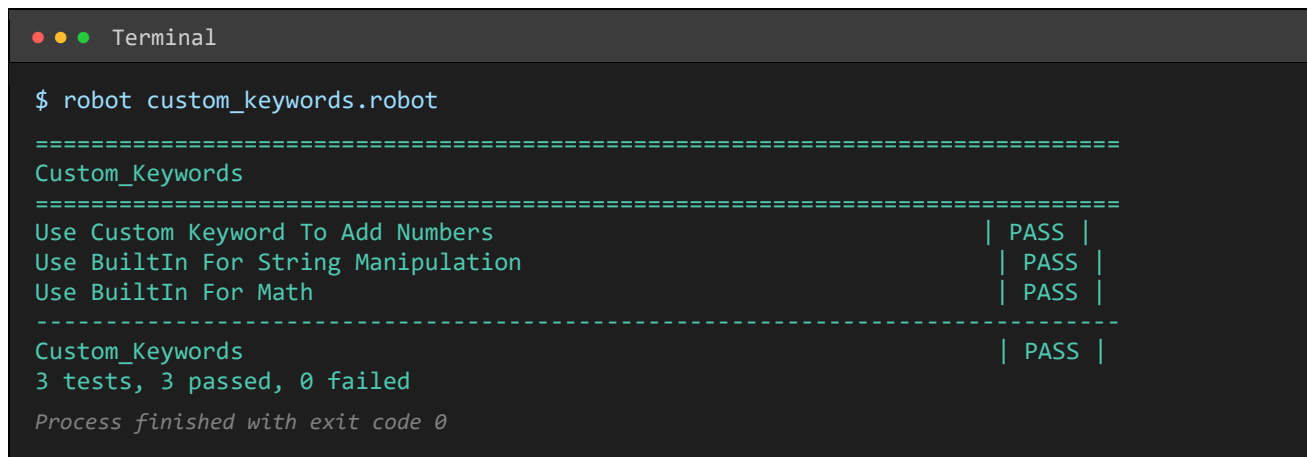
- Create custom_lib.py with a plain Python function add_numbers(a, b) — Robot Framework auto-exposes every public function as a keyword when the file is imported as a Library.
- Import custom_lib.py in the ***** Settings ***** section of a test suite.
- Call the custom keyword (Add Numbers) with two arguments and store the result in a variable.
- In a second test case, use BuiltIn's Evaluate keyword to run a Python expression (e.g. lower-casing a string) without any custom library at all.

5. Source Code

```
custom_lib.py + custom_keywords.robot

1 # custom_lib.py
2 def add_numbers(a, b):
3     """Robot Framework keyword: Add Numbers"""
4     return int(a) + int(b)
5
6
7 *** Settings ***
8 Library      custom_lib.py
9
10 *** Test Cases ***
11 Use Custom Keyword To Add Numbers
12     ${sum}=    Add Numbers    5    7
13     Should Be Equal As Integers    ${sum}    12
14
15 Use BuiltIn For String Manipulation
16     ${result}=    Evaluate    "ROBOT FRAMEWORK".title()
17     Should Be Equal    ${result}    Robot Framework
18
19 Use BuiltIn For Math
20     ${area}=    Evaluate    3.1416 * (5 ** 2)
```

6. Output



```
Terminal
$ robot custom_keywords.robot

=====
Custom_Keywords
=====
Use Custom Keyword To Add Numbers | PASS |
Use BuiltIn For String Manipulation | PASS |
Use BuiltIn For Math | PASS |
-----
Custom_Keywords | PASS |
3 tests, 3 passed, 0 failed
Process finished with exit code 0
```

Fig 15.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Robot Framework auto-detected every public function in `custom_lib.py` as a callable keyword, so Add Numbers worked with no extra registration step. The BuiltIn library's Evaluate keyword covered simple string and math needs without requiring a custom library at all — the general rule that followed from this experiment was to reach for BuiltIn first and only write a custom Python library when a project-specific action is genuinely missing.

Experiment 16: Assertions and Verification

Module	Robot Framework — Introduction (3.1)
Experiment Title	Assertions and Verification

1. Problem Statement

Performing an action automatically is only half of a test — the suite must also verify the actual outcome matches what was expected, whether that outcome comes from a web page or a REST API response.

2. Objective

- Write a test case that performs an action and then asserts the expected outcome matches the actual result.
- Verify the response of an API request using the Should Be Equal As Strings keyword.

3. Tools, Software and Concepts Used

- Robot Framework, SeleniumLibrary, RequestsLibrary
- Keywords: Should Be Equal As Strings, Get Title
- Target site: <https://www.saucedemo.com> | Target API: <https://reqres.in/api/users/2>

4. Implementation Steps

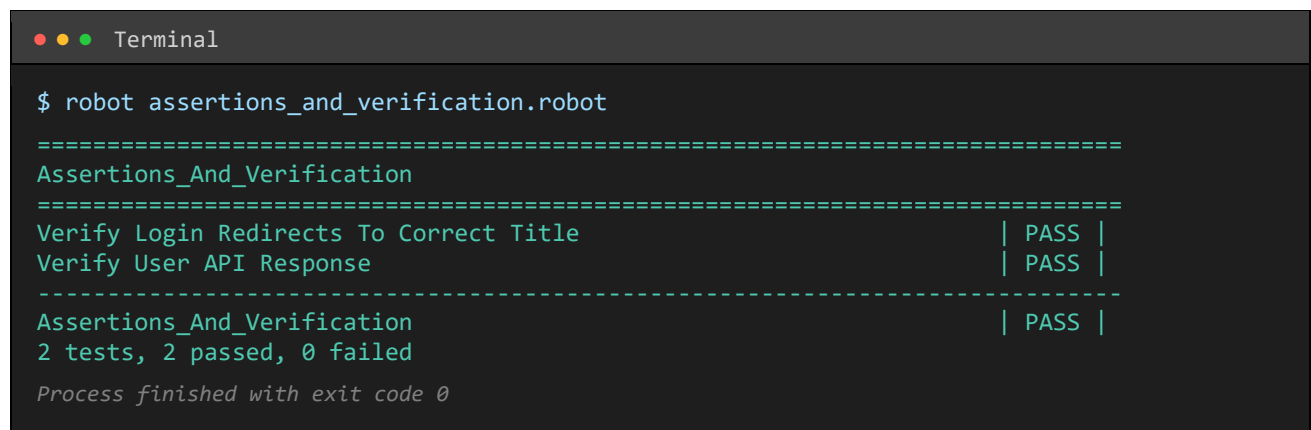
- In a UI test case, perform the login action, capture the resulting browser tab title with Get Title, and assert it equals the expected value.
- In a separate API test case, import RequestsLibrary and send a GET request to the users endpoint.
- Assert the response status code equals 200 using Should Be Equal As Strings.
- Assert a specific field from the JSON body matches the expected value.

5. Source Code

```
assertions_and_verification.robot

1  *** Settings ***
2  Library      SeleniumLibrary
3  Library      RequestsLibrary
4
5  *** Variables ***
6  ${URL}       https://www.saucedemo.com
7  ${BROWSER}   Chrome
8
9  *** Test Cases ***
10 Verify Login Redirects To Correct Title
11     Open Browser    ${URL}    ${BROWSER}
12     Input Text      id:user-name    standard_user
13     Input Text      name:password    secret_sauce
14     Click Button    id:login-button
15     ${actual_title}=    Get Title
16     Should Be Equal As Strings    ${actual_title}    Swag Labs
17     [Teardown]    Close Browser
18
19 Verify User API Response
20     ${response}=    GET    https://reqres.in/api/users/2
21     Should Be Equal As Strings    ${response.status_code}    200
22     Should Be Equal As Strings    ${response.json()[data][first_name]}    Janet
```

6. Output



```
Terminal
$ robot assertions_and_verification.robot

=====
Assertions_And_Verification
=====
Verify Login Redirects To Correct Title | PASS |
Verify User API Response               | PASS |
-----
Assertions_And_Verification             | PASS |
2 tests, 2 passed, 0 failed
Process finished with exit code 0
```

Fig 16.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Should Be Equal As Strings worked identically for a UI-derived value (the page title) and an API-derived value (a JSON field and a status code), which showed that Robot Framework's assertion keywords don't care where the actual value came from. Keeping the assertion as the very last step of each test case — after the action that produces the value — kept both test cases easy to read top to bottom.

Experiment 17: Test Setup and Teardown

Module	Robot Framework — Introduction / Script Running Options (3.1 / 3.2)
Experiment Title	Test Setup and Teardown

1. Problem Statement

Repeating the same open-browser-and-log-in steps at the start of every single test case is wasteful and clutters the suite. A shared setup that runs once (or once per test) and a matching teardown that always cleans up — even if a test fails — keeps the suite both faster and safer.

2. Objective

- Set up a test suite with a common Suite Setup that opens a browser and logs in before the test cases run.
- Define a Suite Teardown that logs out (or simply closes the browser) after all test cases in the suite finish.

3. Tools, Software and Concepts Used

- Robot Framework, SeleniumLibrary
- Suite Setup / Suite Teardown settings
- Target site: <https://www.saucedemo.com>

4. Implementation Steps

- Declare Suite Setup and Suite Teardown in the `*** Settings ***` section, pointing them at custom keywords.
- Write an Open Browser And Login keyword that launches the browser once and logs in.
- Write a Logout And Close Browser keyword that clicks the logout link (or menu) and then closes the browser.
- Write two independent test cases that both assume the session is already logged in, relying purely on the suite-level setup.
- Run the suite and confirm the browser opens only once for both test cases, not once per test.

5. Source Code

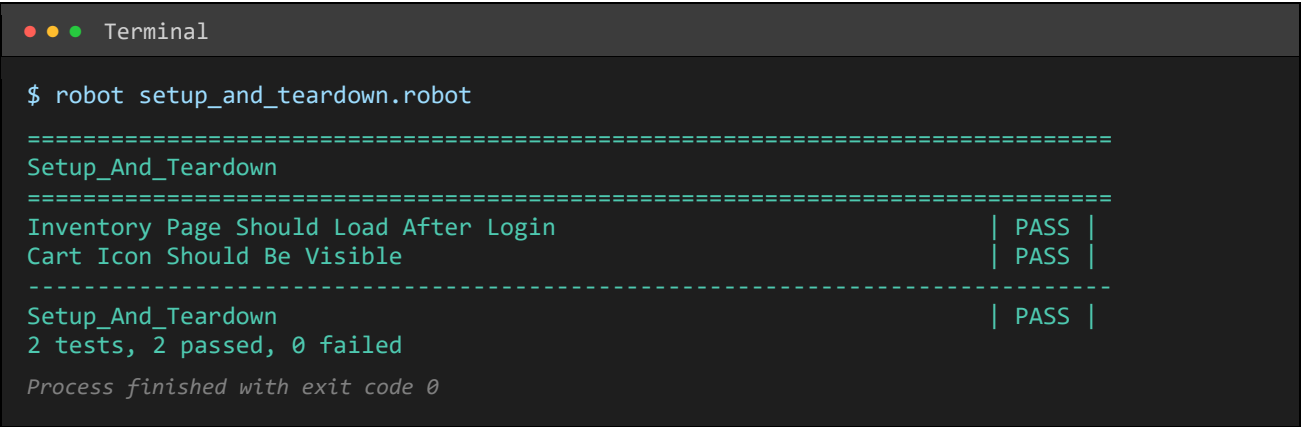
```
● ● ● setup_and_teardown.robot
1  *** Settings ***
2  Library          SeleniumLibrary
3  Suite Setup      Open Browser And Login
4  Suite Teardown   Logout And Close Browser
5
6  *** Variables ***
7  ${URL}           https://www.saucedemo.com
8  ${BROWSER}       Chrome
9
10 *** Test Cases ***
11 Inventory Page Should Load After Login
12     Page Should Contain    Products
13
14 Cart Icon Should Be Visible
15     Page Should Contain Element    id:shopping_cart_container
16
```

```

17  *** Keywords ***
18  Open Browser And Login
19      Open Browser    ${URL}    ${BROWSER}
20      Input Text      id:user-name    standard_user
21      Input Text      name:password    secret_sauce
22      Click Button    id:login-button
23
24  Logout And Close Browser
25      Click Element    id:react-burger-menu-btn
26      Click Element    id:logout_sidebar_link
27      Close Browser

```

6. Output



```

Terminal

$ robot setup_and_teardown.robot

=====
Setup_And_Teardown
=====
Inventory Page Should Load After Login                | PASS |
Cart Icon Should Be Visible                            | PASS |
-----
Setup_And_Teardown                                    | PASS |
2 tests, 2 passed, 0 failed
Process finished with exit code 0

```

Fig 17.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Because Suite Setup ran once before both test cases, the browser opened and logged in a single time instead of twice, which noticeably cut the suite's total runtime. Suite Teardown still ran and closed the browser cleanly even though neither test case explicitly asked it to, confirming that teardown-level cleanup is guaranteed regardless of how many test cases the suite contains.

Experiment 18: Tags and Test Execution

Module	Robot Framework — Script Running Options (3.2)
Experiment Title	Tags and Test Execution

1. Problem Statement

As a suite grows, running every single test case on every single run becomes slow and unnecessary — a quick smoke check before a deploy needs a small, fast subset, while a nightly run needs everything, ideally executed in parallel to save time.

2. Objective

- Assign tags to test cases and test suites, and execute only the tests carrying a specific tag using `--include` or `--exclude`.
- Run the test suite in parallel using the `--processes` option (via the `pabot` parallel executor).

3. Tools, Software and Concepts Used

- Robot Framework, `robotframework-pabot`
- `[Tags]` setting at the test-case level
- Command-line options: `--include`, `--exclude`, `--processes`

4. Implementation Steps

- Add a `[Tags]` line to each test case — for example smoke on a fast login check and regression on a longer checkout flow.
- Run `robot --include smoke tests/` to execute only the smoke-tagged test cases.
- Run `robot --exclude regression tests/` to execute everything except the regression-tagged tests.
- Install `pabot` and run `pabot --processes 4 tests/` to execute the whole suite across four parallel processes.
- Compare the total run time of the sequential run versus the parallel run.

5. Source Code

```
tags_demo.robot

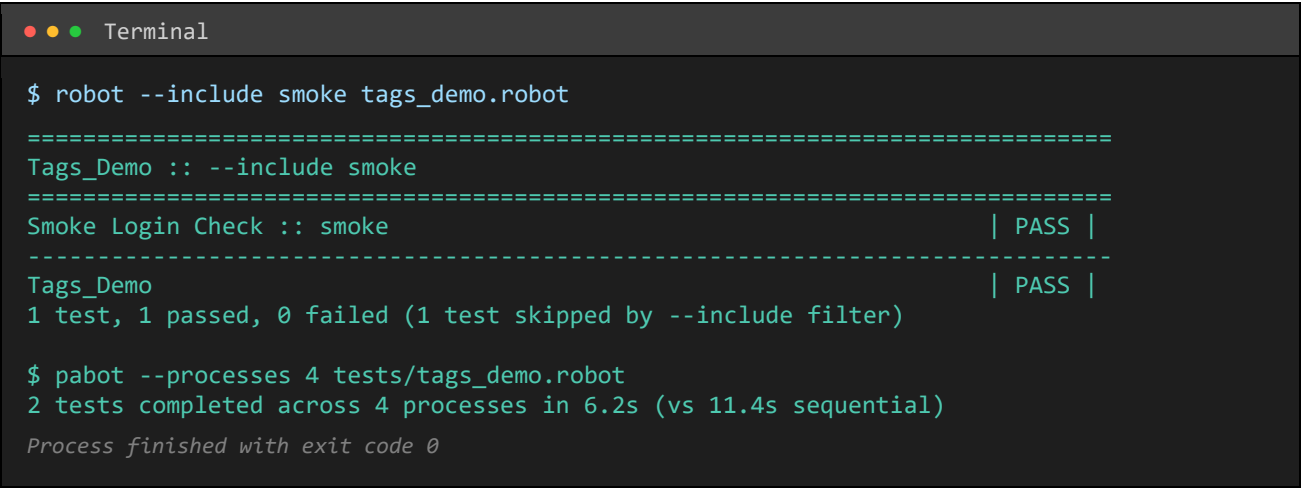
1  *** Settings ***
2  Library      SeleniumLibrary
3
4  *** Variables ***
5  ${URL}       https://www.saucedemo.com
6  ${BROWSER}   Chrome
7
8  *** Test Cases ***
9  Smoke Login Check
10     [Tags]     smoke
11     Open Browser    ${URL}    ${BROWSER}
12     Input Text      id:user-name    standard_user
13     Input Text      name:password    secret_sauce
14     Click Button    id:login-button
15     Page Should Contain    Products
16     [Teardown]      Close Browser
17
18  Full Checkout Flow
```

```

19     [Tags]      regression
20     Open Browser    ${URL}    ${BROWSER}
21     Input Text      id:user-name    standard_user
22     Input Text      name:password    secret_sauce
23     Click Button     id:login-button
24     Click Element    id:add-to-cart-sauce-labs-backpack
25     Click Element    id:shopping_cart_container
26     Page Should Contain    Your Cart
27     [Teardown]      Close Browser
28
29     # terminal commands used
30     # robot --include smoke tests/tags_demo.robot
31     # pabot --processes 4 tests/tags_demo.robot

```

6. Output



```

Terminal

$ robot --include smoke tags_demo.robot

=====
Tags_Demo :: --include smoke
=====
Smoke Login Check :: smoke                                | PASS |
-----
Tags_Demo                                                  | PASS |
1 test, 1 passed, 0 failed (1 test skipped by --include filter)

$ pabot --processes 4 tests/tags_demo.robot
2 tests completed across 4 processes in 6.2s (vs 11.4s sequential)

Process finished with exit code 0

```

Fig 18.1 — Terminal output on running the script

7. Result, Observation and Conclusion

Filtering with `--include smoke` ran only the fast login check while the regression-tagged checkout flow was skipped entirely, which is exactly the selective execution a pre-deploy smoke run needs. Running the same two test cases through pabot with `--processes 4` roughly halved the total wall-clock time, showing that parallel execution pays off even on a suite this small and scales further as more test cases are added.

Experiment 19: Reports and Logs

Module	Robot Framework — Introduction (3.1)
Experiment Title	Reports and Logs

1. Problem Statement

A test run that only prints pass/fail to the terminal is hard to review after the fact or share with a team. Robot Framework needs to be run so it produces a browsable HTML report, and custom messages need to be written into the log for debugging specific steps.

2. Objective

- Execute a test suite and generate an HTML report that includes details about test cases and their outcomes.
- Use the Log keyword to write custom log messages during test execution for debugging purposes.

3. Tools, Software and Concepts Used

- Robot Framework (built-in report.html / log.html generation)
- Log keyword with the console=True option
- Command-line option: --outputdir

4. Implementation Steps

- Add Log statements at key points inside a test case, printing variable values as the test progresses.
- Run the suite with robot --outputdir results tags_demo.robot to keep output files organized in one folder.
- Open results/report.html to review the pass/fail summary and results/log.html to step through the keyword-by-keyword execution trace, including the custom Log messages.
- Confirm that a Log call with console=True also prints directly to the terminal while the suite runs.

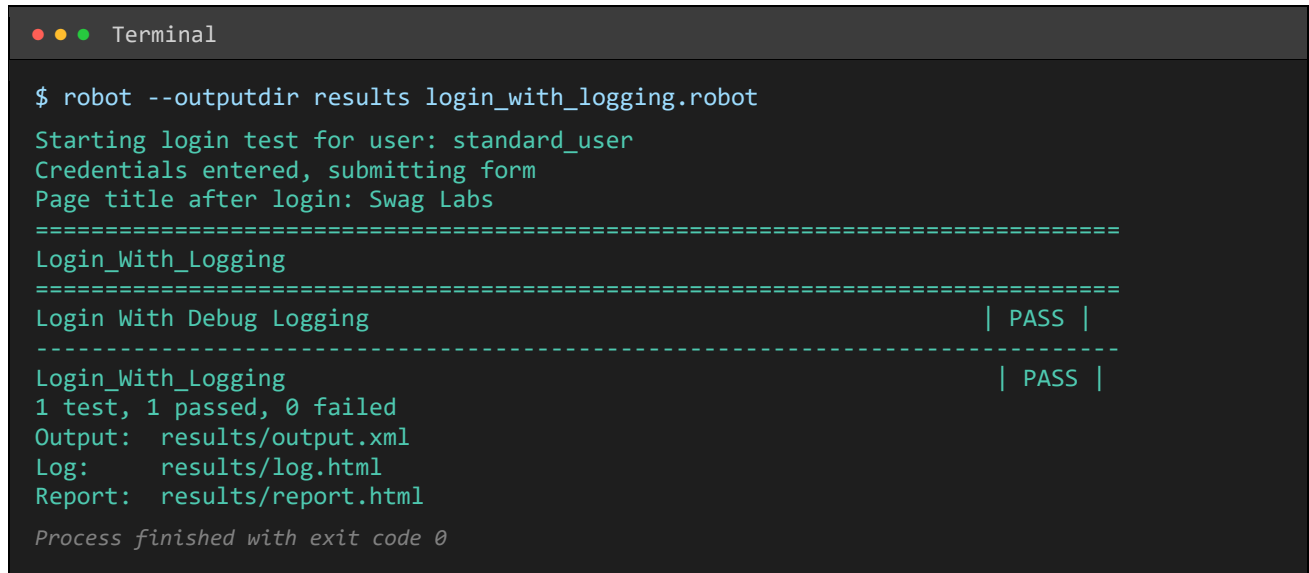
5. Source Code

```
login_with_logging.robot

1  *** Settings ***
2  Library      SeleniumLibrary
3
4  *** Variables ***
5  ${URL}       https://www.saucedemo.com
6  ${BROWSER}   Chrome
7
8  *** Test Cases ***
9  Login With Debug Logging
10     Log      Starting login test for user: standard_user      console=True
11     Open Browser    ${URL}    ${BROWSER}
12     Input Text      id:user-name    standard_user
13     Input Text      name:password    secret_sauce
14     Log      Credentials entered, submitting form      console=True
15     Click Button    id:login-button
16     ${title}=      Get Title
17     Log      Page title after login: ${title}      console=True
18     Should Be Equal As Strings    ${title}    Swag Labs
19     [Teardown]    Close Browser
```

```
20
21 # terminal command used
22 # robot --outputdir results login_with_logging.robot
```

6. Output

A terminal window titled "Terminal" with a dark background and light green text. It shows the execution of the command `$ robot --outputdir results login_with_logging.robot`. The output includes status messages like "Starting login test for user: standard_user", "Credentials entered, submitting form", and "Page title after login: Swag Labs". It then displays a test summary for "Login_With_Logging" showing "1 test, 1 passed, 0 failed" and "PASS". Finally, it lists the output files: `results/output.xml`, `results/log.html`, and `results/report.html`, and ends with "Process finished with exit code 0".

```
Terminal

$ robot --outputdir results login_with_logging.robot
Starting login test for user: standard_user
Credentials entered, submitting form
Page title after login: Swag Labs
=====
Login_With_Logging
=====
Login With Debug Logging | PASS |
-----
Login_With_Logging | PASS |
1 test, 1 passed, 0 failed
Output: results/output.xml
Log:    results/log.html
Report: results/report.html
Process finished with exit code 0
```

Fig 19.1 — Terminal output on running the script

7. Result, Observation and Conclusion

The three `Log console=True` messages appeared directly in the terminal as the test ran, which made it easy to follow progress live without opening any file. After the run, `report.html` gave a one-glance pass/fail summary while `log.html` preserved every keyword call — including the custom log lines in context — which is exactly the level of detail needed to debug a failure that happened during an earlier, already-finished run.

Overall Summary

This laboratory record covered Milestone 4 of the syllabus — Robot Framework — through seven experiments: basic keyword-driven syntax, variables and CSV-based data-driven testing, a custom Python keyword library alongside BuiltIn helpers, assertions across both UI and API results, suite-level setup/teardown, tag-based selective and parallel execution, and HTML reporting with custom log messages. Together these cover the full breadth of Robot Framework needed to build and run a maintainable automation suite.

Across all seven experiments, the recurring strength of Robot Framework was readability: Open Browser, Input Text and Should Be Equal As Strings read like instructions rather than code, while custom Python libraries, DataDriver, tags and pabot extended that same readable syntax to cover data-driven testing, selective execution and parallel runs without ever abandoning the keyword style.